Internship Report

Tracking and Segmenting Objects in Natural Environments

Pugh Joshua

Internship Period: April 14, 2025 – July 18, 2025

Laboratoire Mathématiques, Image et Applications
La Rochelle Université

Submitted on:
June 26, 2025

Supervisors:
Péteri Renaud
Mascarilla Laurent

Acknowledgements

I would like to express my deepest gratitude to Mr. Péteri for his warm welcome and support during my internship at the MIA laboratory. His availability and insightful guidance significantly enriched my experience and contributed to my professional growth.

I extend my sincere thanks to my internship supervisor, Mr. Mascarilla, for his oversight and support throughout this period. I am also immensely grateful to La Rochelle Université and the MIA laboratory for providing me with this invaluable opportunity, which allowed me to deepen my knowledge and apply the skills acquired during my Masters in computer science.

Additionally, I wish to acknowledge the LABRI team from Bordeaux University, with special thanks to Jenny Benois-Pineau, Ander Echezarreta-Arruti, and Lucas Bardisbanian, for their collaboration and support during my remote work with them. Their contributions were instrumental in the success of this internship.

This experience has been profoundly rewarding and has further solidified my professional aspirations. Thank you all for making this opportunity possible.

Contents

1 Introduction **1**
1.1 Internship Overview . . . 1
1.2 Project Context and Objectives . . . 2

2 Datasets **3**
2.1 Grasping In The Wild (GITW) Dataset . . . 3
2.2 Cylinder-Clothpin Dataset . . . 3

3 Metrics and Evaluation **4**
3.1 Intersection over Union (IoU) . . . 4
3.2 Precision . . . 5

4 Point Extraction from Object Masks **6**
4.1 Grid-Based Point Extraction . . . 6
4.2 Random Sampling Point Extraction . . . 7
4.3 Trade-Offs and Evaluation . . . 8

5 Object Tracking Methodology **9**
5.1 Overview . . . 9
5.2 Key Considerations . . . 9

6 Homography **10**
6.1 Computation Method . . . 10
6.2 Limitations . . . 11
6.3 Local vs. Global Keypoints . . . 11
6.4 Evaluation . . . 12

7 Correction **13**
7.1 Contour Detection Methods . . . 13
7.1.1 GrabCut . . . 13
7.1.2 Active Contour . . . 13
7.1.3 Chan-Vese . . . 14
7.1.4 Morphological Geodesic Active Contour . . . 14
7.2 Evaluation and Results . . . 15

8 Possible Improvements **16**
8.1 Improving Accuracy . . . 16
8.2 Detecting Contour Detection Failures . . . 17

9 A New Approach **18**
9.1 Segment Anything Model 2 (SAM2) . . . 18
9.1.1 Architecture and Key Components . . . 18
9.1.2 Modes of Operation . . . 19
9.1.3 Video Tracking with Memory . . . 19

10 Integrating SAM2 into the Tracking Pipeline **20**
10.1 SAM2 as a Replacement for Contour Detection 20
10.1.1 Method 20
10.1.2 Evaluation 21
10.2 SAM2 as a Full Replacement 22
10.2.1 Method 22
10.2.2 Evaluation 23

11 Conclusion **24**

1 Introduction

1.1 Internship Overview

This report presents my work conducted on the I-WRIST project during my internship at the *Laboratoire Mathématiques Image Applications* (MIA), in collaboration with the *Laboratoire Bordelais de Recherche en Informatique* (LABRI), *Institut de Neurosciences Cognitives et Intégratives d'Aquitaine* (INCIA), Pollen Robotics and UGECAM, commencing on April 1, 2025. The MIA Laboratory, established in 2008, specializes in applied mathematics, image processing, and computer science, and is a member of the LUDI Institute (Littoral Urbain Durable Intelligent) at La Rochelle Université, as well as the FREDD, MIRES, and MARGAUX research federations. Directed by Catherine Choquet, MIA comprises a multidisciplinary team of 14 researchers, with expertise in signal processing, computer vision, and machine learning.

My work at MIA focused on developing a gaze-based object tracking system to enhance the functionality of upper limb neuro-prostheses, with a particular emphasis on the object tracking phase. My internship was conducted on-site at the MIA Laboratory in La Rochelle, primarily under the supervision of M. Péteri, however there was also a strong collaboration with the LABRI team performed remotely. A approximate overview of the different phases of my internship is provided bellow in Figure 1, with the red line indicating the date when this report was writen.

Figure 1: Chart showing the approximate timeline and phases of the internship project.

1.2 Project Context and Objectives

The primary objective of this project is to empower individuals with above-elbow amputations to control robotic prosthetic arms using gaze data. Traditional myoelectric control systems, which rely on residual muscle signals, often fall short in managing the multiple degrees of freedom required for complex tasks. To address this limitation, computer vision techniques, leveraging gaze data captured by wearable devices, provide a promising solution for identifying and tracking objects of interest for grasping tasks. This approach aims to improve the precision and intuitiveness of prosthetic control, thereby enhancing the quality of life for amputees.

The system is structured around four key phases: initial object detection, object tracking, 3D object shape extraction, and prosthetic movement. Data collection was performed using advanced wearable devices, specifically a Microsoft HoloLens 2 and a Tobii Pro Glasses 2. Both devices capture RGB video and gaze data, enabling precise identification of the user's focus. Additionally, the HoloLens 2 provides depth information, which is critical for reconstructing the 3D structure of the target object, thereby facilitating accurate shape extraction and prosthetic control.

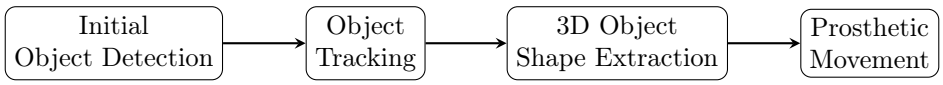


Figure 2: Overview of the gaze-based object tracking pipeline.

The pipeline, illustrated in Figure 2, consists of the following stages:

- **Initial Object Detection**: This stage identifies the target object the user intends to grasp by analyzing gaze fixation data. Gaze points, captured via wearable eye-tracking devices, serve as prompts to guide the detection process. Further details on this methodology can be found in the referenced study [1].
- **Object Tracking**: Once the target object is identified, real-time tracking is employed to maintain a consistent focus on the object across video frames. This phase, which forms the core of my internship, is detailed extensively in subsequent sections of this report.
- **3D Object Shape Extraction**: Using the object mask generated during tracking, the system samples a set of points from the masked area on a depth map provided by the HoloLens 2. These points are used to reconstruct the 3D shape of the object, capturing its spatial orientation and structure for accurate prosthetic control.
- **Prosthetic Movement**: The extracted 3D shape informs the control algorithms of the robotic arm, enabling precise movements for grasping and manipulating the target object.

To support the development and evaluation of the object tracking system, diverse datasets were employed. The following sections detail these datasets, including the Grasping In The Wild (GITW) and Cylinder-Clothpin datasets, along with the performance metrics used to assess the system's effectiveness.

2 Datasets

This section describes the two datasets utilized in this project: the Grasping In The Wild (GITW) dataset and the Cylinder-Clothpin dataset. These datasets provide diverse scenarios for developing and evaluating the entire pipeline.

2.1 Grasping In The Wild (GITW) Dataset

The **Grasping In The Wild** (GITW) dataset is a comprehensive collection of egocentric video sequences capturing everyday objects, such as plates, bowls, and frying pans, in naturalistic, cluttered environments like kitchens. This dataset poses significant challenges for object tracking due to its complex backgrounds and varying lighting conditions, making it an ideal testbed for real-world applications.

Frying pan example | Mug example | Plate example

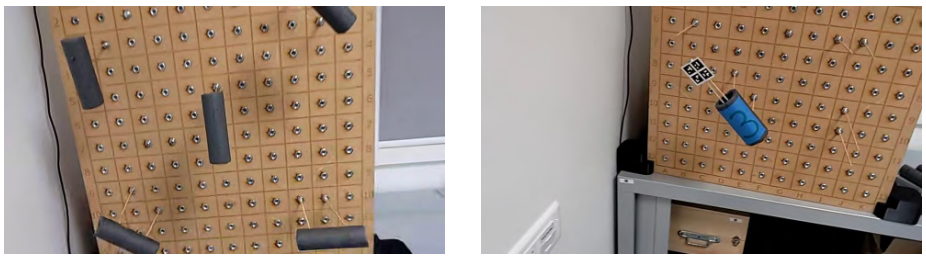
Figure 3: Sample frames from the GITW dataset.

The data was recorded using the *Tobii Pro Glasses 2*, a wearable eye-tracking device that captures RGB video at 25 Hz and gaze fixation points at 50 Hz. While the Tobii Pro Glasses 2 lack depth sensors, this does not impact the object tracking phase, which relies solely on RGB video and gaze data to generate object masks. The dataset includes manually annotated binary masks for the objects of interest in each frame, providing ground truth segmentations that facilitate the benchmarking of object tracking algorithms. With 16 object classes and over 13,000 annotated frames, the GITW dataset allows for robust evaluation of tracking performance in diverse, real-world scenarios.

2.2 Cylinder-Clothpin Dataset

The Cylinder-Clothpin dataset provides a controlled environment designed as an initial step for developing and refining object tracking algorithms, as well as evaluating the 3D object shape extraction phase of the pipeline. It features simple objects—cylinders and clothespins—placed in uncluttered settings, offering

an easier challenge compared to the complex scenarios in the GITW dataset. These objects can be equipped with tags that provide ground truth data for their orientation and shape, enabling accurate assessment of both tracking performance and the system's ability to reconstruct 3D structures using depth information from the Microsoft HoloLens 2.



Cylinder example Cylinder with tag

Figure 4: Sample frames from the Cylinder-Clothpin dataset.

This dataset's simplicity makes it a valuable resource for early-stage algorithm development, allowing for the testing and optimization of tracking techniques in a controlled setting before applying them to the more challenging real-world conditions of the GITW dataset. Additionally, the ground truth tags support precise validation of 3D shape extraction, ensuring the pipeline's robustness. The Cylinder-Clothpin dataset thus serves as a complementary resource, bridging foundational development and real-world application.

Together, these datasets provide a robust foundation for developing and evaluating the gaze-based object tracking system, addressing both real-world challenges and controlled validation requirements.

3 Metrics and Evaluation

This section outlines the evaluation metrics used to assess the performance of the gaze-based object tracking system, focusing on the object tracking and 3D object shape extraction phases of the pipeline. The primary metrics employed are Intersection over Union (IoU) and precision, which provide robust measures of segmentation accuracy and tracking reliability across the Grasping In The Wild (GITW) and Cylinder-Clothpin datasets.

3.1 Intersection over Union (IoU)

Intersection over Union (IoU), also known as the Jaccard Index, is a widely adopted metric for evaluating the quality of object segmentation and tracking in computer vision tasks. It measures the overlap between the predicted binary mask (M) and the ground truth mask ($\bar{M}$) for a given video frame f, as defined by the following equation:

$$\text{IoU}_f = \frac{|M \cap \bar{M}|}{|M \cup \bar{M}|}$$

Here, $|M \cap \bar{M}|$ represents the number of pixels in the intersection of the predicted and ground truth masks, and $|M \cup \bar{M}|$ denotes the number of pixels in their union. The IoU score ranges from 0 to 1, with a value of 1 indicating perfect overlap and 0 indicating no overlap.

For a video sequence of N frames, the mean IoU (mIoU) is computed as the average IoU across all frames:

$$\text{mIoU} = \frac{1}{N} \sum_{i=1}^{N} \text{IoU}_{f_i}$$

In the context of this project, IoU is used to evaluate the accuracy of the object masks generated during the tracking phase. For the GITW dataset, which contains manually annotated ground truth masks for 16 object classes in cluttered environments, IoU provides a precise measure of how well the system tracks objects under challenging conditions. For the Cylinder-Clothpin dataset, IoU assesses the initial tracking performance in a controlled setting, ensuring the algorithm's reliability before scaling to more complex scenarios.

3.2 Precision

Precision is employed as a complementary metric to evaluate the reliability of the points generated from the object mask for 3D shape extraction. In this project, the 3D shape estimation model requires a set of N points sampled from the masked area on a depth map, representing the object's surface. Precision is defined as the ratio of predicted points that correctly lie within the ground truth object mask to the total number of predicted points:

$$\text{Precision} = \frac{\text{Number of predicted points inside } M}{\text{Total number of predicted points}}$$

Here, the numerator counts the points from the predicted set that fall within the ground truth mask $\bar{M}$, and the denominator is the total number of points (N) generated from the predicted mask. Precision ranges from 0 to 1, with higher values indicating that a greater proportion of the sampled points accurately represent the object's surface.

This metric is critical for ensuring the quality of the input points used in 3D shape reconstruction as points being outside the object could lead to a degradation of the 3D shape estimation.

4 Point Extraction from Object Masks

This section details the methods developed during my internship to extract points from object masks generated during the tracking phase. These points, sampled from the masked area on the depth map, are critical inputs for the 3D reconstruction of the object. Extracting points is a non-trivial task, as it requires balancing uniform coverage of the object with the capture of fine details, while ensuring compatibility with the depth data provided by the Microsoft HoloLens 2. Two approaches were explored: a grid-based method for even distribution and a random sampling method for variability across frames. Each method presents unique advantages and challenges, which are discussed below.

4.1 Grid-Based Point Extraction

The first approach involves sampling points from the object mask using a regular grid to ensure an even distribution across the object's surface. The process is as follows:

1. **Mask Identification**: The binary mask M for a given frame, generated during the object tracking phase, identifies the pixels belonging to the object of interest.

2. **Grid Overlay**: A uniform grid is overlaid on the bounding box of the mask, with a predefined spacing between grid points in both horizontal and vertical directions.

3. **Point Selection**: Points are selected where grid intersections fall within the mask (i.e., where $M(x, y) = 1$). This yields a set of N points, where N is determined by the grid density and the mask's area.

4. **Depth Mapping**: Each selected point (x, y) is mapped to the corresponding depth value from the HoloLens 2 depth map.

The grid-based method ensures a consistent and evenly distributed set of points, which is advantageous for maintaining uniform coverage of the object's surface. This is particularly beneficial for objects with regular shapes, such as those in the Cylinder-Clothpin dataset, where uniform sampling can accurately represent the object's geometry. Additionally, the deterministic nature of the grid simplifies implementation and ensures repeatability across frames.

However, this approach has limitations. The fixed grid may miss fine details in complex or irregularly shaped objects, such as

those in the Grasping In The Wild (GITW) dataset (e.g., frying pans with handles). If the grid spacing is too coarse, small or thin regions of the object may be underrepresented, creating "blind spots" in the point cloud. Conversely, a very fine grid increases computational cost and may produce redundant points in larger, uniform areas. The grid's static nature also means that certain areas of the object are consistently sampled, potentially overlooking subtle variations in shape or texture over time.

4.2 Random Sampling Point Extraction

The second approach employs random sampling to generate points within the object mask, aiming to mitigate the fixed-pattern limitations of the grid-based method. The process is as follows:

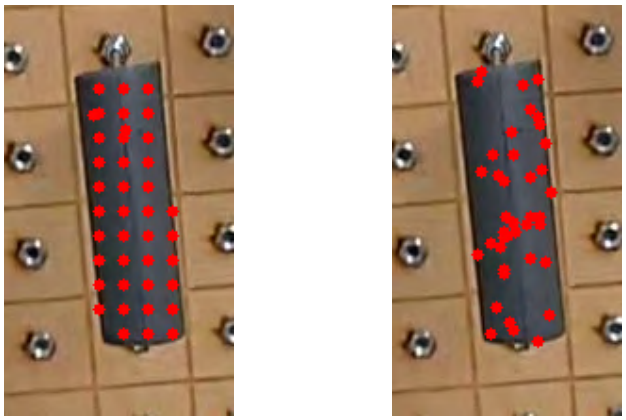
1. **Mask Identification**: As in the grid-based method, the binary mask M defines the object's pixel region.
2. **Random Point Generation**: A set of N points is randomly sampled from the pixels where $M(x, y) = 1$. This is typically achieved using a uniform random distribution over the mask's non-zero pixels.
3. **Depth Mapping**: Each randomly sampled point (x, y) is mapped to its corresponding depth value from the HoloLens 2 depth map.

Random sampling ensures variability in point placement across frames, reducing the risk of persistent blind spots in the point cloud. Over a video sequence, this approach is likely to capture a broader range of object details, as different regions are sampled in each frame. This is particularly advantageous for the GITW dataset, where objects in cluttered environments may have complex shapes or occlusions that benefit from diverse sampling patterns.

However, the random nature of this method introduces challenges. In any single frame, the point distribution may be uneven, with some areas of the object oversampled and others undersampled. This variability can lead to inconsistent 3D shape reconstructions, especially for smaller objects or thin structures where precise point placement is critical. To mitigate this, a sufficient number of points (N) must be sampled to ensure adequate coverage, but this increases computational overhead. Additionally, the randomness makes it difficult to guarantee repeatability, which can complicate debugging and evaluation.

4.3 Trade-Offs and Evaluation

Both methods were implemented, but the evaluation was very limited due to the internship's focus on object tracking. Further research is needed to fully assess the optimal point extraction approach.



Grid-based. Random sampling.

Figure 5: Comparison of grid-based and random sampling point extraction methods on a cylinder from the Cylinder-Clothpin dataset.

Figure 5 shows the grid-based and random sampling methods applied to a cylinder from the Cylinder-Clothpin dataset. The grid-based method provides even point distribution, while the random sampling method offers variable distribution.

The choice between these methods depends on the object's complexity and the needs of the 3D reconstruction model. Future work could explore hybrid methods, such as adaptive grids that adjust based on object shape or weighted random sampling to focus on key areas. The following sections detail the object tracking methodologies, including homography and SAM2-based approaches, which generate the masks used for point extraction.

5 Object Tracking Methodology

5.1 Overview

The object tracking process begins with an initial mask, denoted M_0, provided by the initial object detection step. The objective is to generate a distribution of points on the object of interest for each frame, which requires computing an accurate mask per frame from which points can be extracted.

The tracking pipeline uses the previous frame's mask M_{n-1} to compute an approximate mask MA_n for the current frame via homography, which provides a robust approximation in most scenarios. This approximate mask is then refined through a correction step to produce the final mask M_n, which is used for tracking in the subsequent frame and for point extraction. The process is iterative, repeating for each frame to maintain tracking continuity.

The methodology is illustrated in the following diagram:

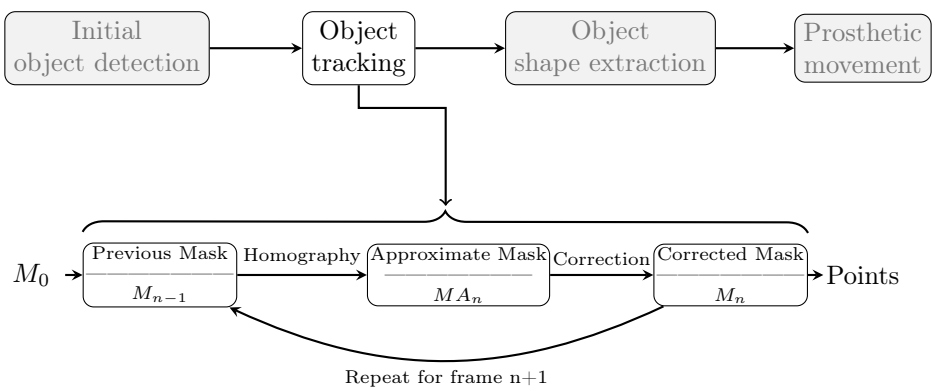


Figure 6: Overview of the object tracking methodology.

5.2 Key Considerations

The proposed methodology raises several critical questions that guide the subsequent sections:

- What is the optimal approach for computing the homography to generate the approximate mask MA_n?
- Is the correction step essential for maintaining tracking accuracy?
- How frequently should the correction step be applied to balance accuracy and computational efficiency?
- What is the most effective method for correcting the approximate mask to obtain the final mask M_n?

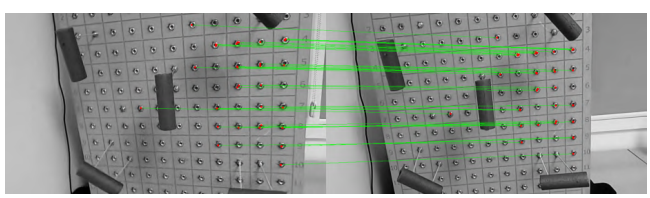
6 Homography

Homography is employed as a lightweight method to compute an approximate mask MA_n from the previous mask M_{n-1}, facilitating object contour detection by starting with a close approximation. This approach reduces the frequency of computationally expensive correction steps, allowing the mask to be updated quickly for most frames.

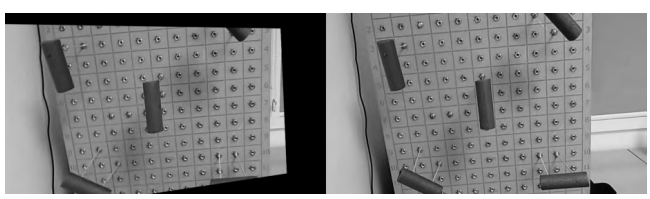
6.1 Computation Method

To compute the homography between consecutive frames, corresponding points between the previous mask M_{n-1} and the current frame are identified using the ORB (Oriented FAST and Rotated BRIEF) [10] keypoint detector and descriptor, known for its efficiency and robustness in real-time applications. The process is as follows:

1. Detect ORB keypoints and compute descriptors in the previous frame and in the current frame.
2. Match descriptors between the two frames using a matcher (e.g., brute-force or FLANN-based).
3. Select the best matches and extract the coordinates of the matched keypoints.
4. Compute the homography matrix using OpenCV's `findHomography` function with RANSAC to enhance robustness against outliers.
5. Warp the previous mask M_{n-1} onto the current frame using the homography matrix to produce the approximate mask MA_n.



Matching keypoints between frames.



Warping previous frame.

Figure 7: Illustration of homography-based warping.

This method leverages ORB's distinctive features to establish reliable correspondences, improving homography accuracy under conditions like object rotation or moderate occlusion.

6.2 Limitations

Homography assumes a planar transformation, which may not hold in 3D scenes, leading to potential errors. Additionally, it relies solely on transformation estimation without object detection, causing drift over time. The method also assumes minimal object movement between frames, which may not always be valid.

6.3 Local vs. Global Keypoints

To address the planar assumption limitation, using only local keypoints to the object (within a radius r pixels around the object's mask) was investigated to determine if it could reduce the error compared to using keypoints from the entire image. The hypothesis is that local keypoints could be more planar than the entire image, minimizing errors caused by homography being a planar transformation. Tests were conducted on the frying pan object from the GITW dataset, with the following mean IoU results for different r values (local area radius in pixels) and the full image (r = None):

Radius (r)	Mean IoU
50	0.8377
100	0.8377
150	0.8390
200	0.8469
250	0.8542
300	0.8475
None (Full Image)	0.8403

Table 1: Mean IoU for homography using local keypoints (various radii) vs. global keypoints (full image) on the frying pan object.

The results show that using local keypoints with $r = 250$ yields the highest mean IoU (0.8542), slightly outperforming the full-image approach (0.8403). However, the differences across $r = 250$ and the full-image case are minimal, suggesting that restricting keypoints to a local area does not significantly improve homography accuracy in this context.

It is also important to note that restricting keypoints to a local region can introduce issues if there are not enough distinctive features within the specified radius. In such cases, the homography estimation may fail or become unstable due to an insufficient number of reliable matches, especially for small or textureless objects.

6.4 Evaluation

An evaluation of the homography transformation was performed, frame by frame, on the PlatePlace1Subject2 sequence from the GITW dataset, which contains a plate object. The evaluation focused on the IoU of the approximate mask MA_n against the ground truth mask $\bar{M}_n$ across frames, and using keypoints from the entire image. The results are presented in Figure 8.

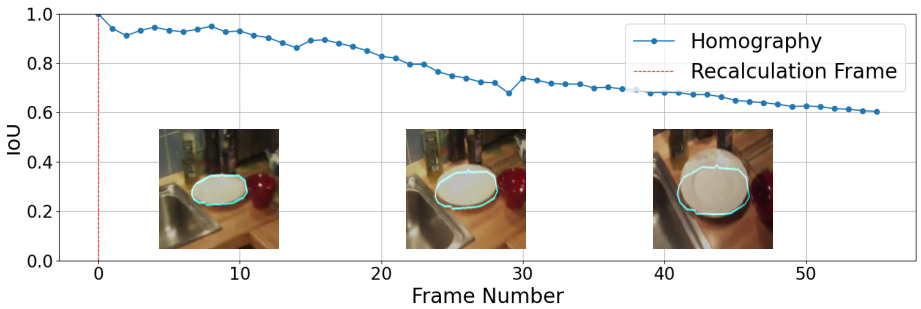


Figure 8: IoU without correction on *PlatePlace1Subject2*

The observed drift indicates that homography alone is insufficient for sustained tracking accuracy, necessitating a correction step. To evaluate the efficiency of a correction step, corrections were simulated by resetting the mask to the ground truth every N frames, $N = 10$ chosen for this test:

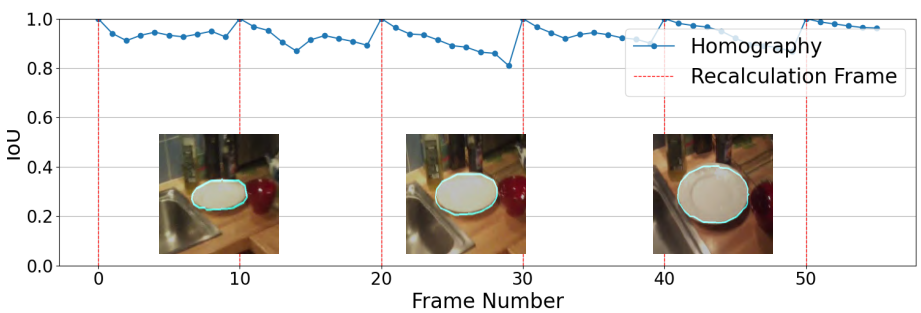


Figure 9: IoU with correction every 10 frames on *PlatePlace1Subject2*

The results demonstrate that correcting the mask every 10 frames is able to maintain high accuracy while limiting the computational overhead of the correction step, which could be resource-intensive.

7 Correction

As shown previously, the correction step is essential to eliminate the drift in the approximate mask MA_n derived from homography, ensuring the mask remains aligned with the object of interest. While perfection is not required, a closer approximation enhances tracking reliability for neuro-prosthetic control.

Initial efforts focused on contour detection methods implemented in the scikit-image and OpenCV libraries, chosen for their simplicity and lack of requirement for extensive training. However, these methods proved sensitive to environmental changes, such as lighting variations, and struggled in low-gradient scenarios, such as a black frying pan in a dark kitchen or a white plate in a bright kitchen.

7.1 Contour Detection Methods

Contour detection aims to delineate object boundaries in an image, typically relying on gradients or region properties. While effective in controlled settings, these methods are sensitive to scene variations, such as changes in lighting caused by movement, which can disrupt performance. Additionally, gradient-based methods require distinct intensity differences, which may be absent in low-contrast environments, leading to unreliable segmentation.

Several contour detection algorithms were evaluated:

7.1.1 GrabCut

GrabCut [9], as implemented in OpenCV, is an iterative algorithm that uses graph cuts to separate the foreground (object of interest) from the background. It requires an initial bounding box, which can be derived from MA_n. The process is as follows:

1. Initialize with a bounding box around the object or a rough mask.
2. Construct a graph where pixels are nodes, and edges represent the cost of assigning pixels to the foreground or background based on color models.
3. Use Gaussian Mixture Models to estimate the likelihood of each pixel belonging to the foreground or background.
4. Iteratively refine the segmentation by minimizing an energy function using graph cuts.
5. Output a binary mask separating the object from the background.

7.1.2 Active Contour

Active Contour [5], or snakes, as implemented in scikit-image, evolves a curve to fit object boundaries using image gradients. The process involves:

1. Initialize a curve (snake) around the object, typically based on the boundary of MA_n.
2. Compute image gradients to identify edges, often using a smoothed version of the image (e.g., Gaussian filtering).
3. Evolve the curve by minimizing an energy function that balances internal forces (curve smoothness and elasticity) and external forces (gradient attraction).
4. Converge the curve to the object boundaries, producing a binary mask.

This method requires careful parameter tuning (e.g., alpha for smoothness, beta for rigidity) to avoid converging to incorrect boundaries in noisy or low-contrast images.

7.1.3 Chan-Vese

Chan-Vese [2][3][4], implemented in scikit-image, is a region-based segmentation method that does not rely on edge gradients, making it suitable for low-contrast scenes. It operates as follows:

1. Initialize a contour, often a checkerboard pattern or a shape derived from MA_n.
2. Define an energy function based on intensity differences between the regions inside and outside the contour.
3. Evolve the contour using level-set methods to minimize the energy function, separating the object from the background.
4. Output a binary mask of the object.

The method requires tuning parameters like the regularization term (`mu`) and iteration count to balance region homogeneity and contour smoothness.

7.1.4 Morphological Geodesic Active Contour

Morphological Geodesic Active Contour [7], implemented in scikit-image, is a level-set-based method that combines morphological operations with geodesic active contours to delineate object boundaries. It is particularly effective for refining an initial mask in complex scenes. The process is as follows:

1. Initialize with an initial mask or contour, typically derived from MA_n.
2. Pre-process the image to compute a gradient or edge map (e.g., using Sobel filtering or inverse Gaussian gradient) to guide the contour evolution.
3. Evolve the contour using a level-set approach, where morphological operations (dilation and erosion) are applied iteratively to minimize a geodesic energy function that balances adherence to image features and contour smoothness.

4. Output a binary mask representing the object boundaries.

This method requires tuning parameters such as the number of iterations, threshold for convergence, and smoothing factors to prevent over-segmentation or under-segmentation in noisy or cluttered environments.

7.2 Evaluation and Results

Among the tested methods, GrabCut proved the most robust to varying conditions, making it the focus of further evaluation. To assess performance, contour detection was applied to correct the approximate mask every N frames, analogous to the homography evaluation, but using the contour detection output instead of ground truth masks. Tests were conducted with $N = 10$:

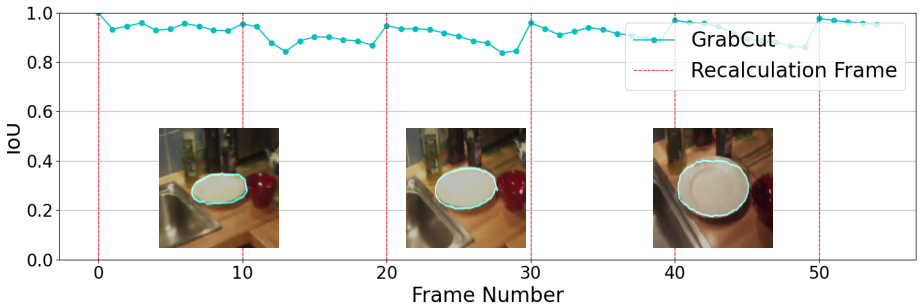


Figure 10: IoU with correction every 10 frames on *PlatePlace1Subject2*

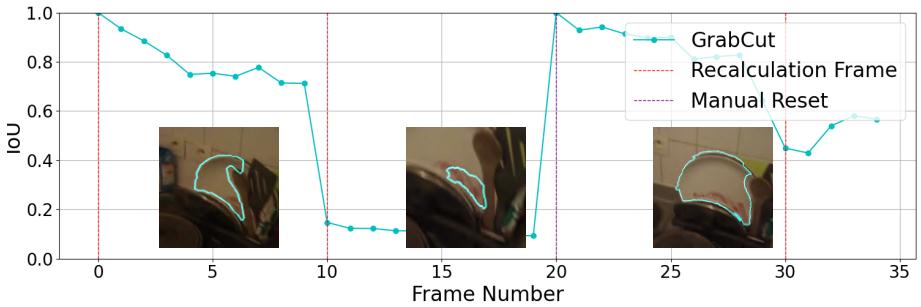


Figure 11: IoU with correction every 5 frames on *PlatePlace1Subject2*

When successful, GrabCut produced accurate masks with a mean IoU of 0.92, closely aligning with object boundaries (see Figure 10). However, its performance was inconsistent in complex or cluttered environments with occlusions or in low-light conditions. As shown in Figure 11, its performance was so poor that it completely lost the object of interest at frame 20, requiring a manual reset. This resulted in a mean IoU of 0.58, even with the manual reset.

A significant limitation is the lack of a confidence score to indicate the reliability of the generated mask in real-time settings, making it challenging to detect failures without ground truth comparison. This inconsistency and lack of reliability prompted exploration of more robust correction methods, such as SAM2, as discussed in subsequent sections.

8 Possible Improvements

To enhance the contour detection process, an attempt was made to improve the accuracy of generated masks, and a method was developed to detect contour detection failures, ensuring reliability in real-time applications.

8.1 Improving Accuracy

An investigation was conducted using histogram analysis of the masked area to improve contour detection accuracy. The hypothesis assumed that, given objects are relatively monochromatic, the histogram of pixel intensities within the mask should remain consistent across frames. Significant changes in the histogram could indicate inaccuracies in the mask, enabling refinement of the contour.

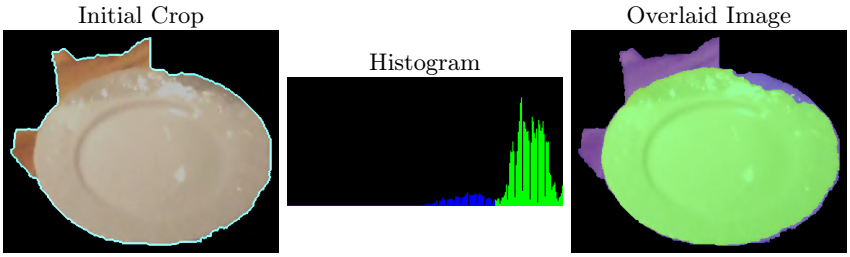


Figure 12: Histogram analysis for mask refinement.

Note: This example represents a best-case scenario where the object is monochrome on a simple background.

It is visible that the different peaks in the histogram represented by blue and green, correspond well to the object of interest (green) and the background mistakenly included in the segmentation (blue).

However, this approach was largely ineffective. The limited dynamic range of the Tobii Pro Glasses 2 and HoloLens cameras resulted in compressed histograms, making it difficult to detect meaningful changes. Additionally, the assumption of monochromatic objects does not hold for many objects in the GITW dataset, which exhibit varied textures and colors. Even when histogram changes were detected, removing newly introduced regions (e.g., background areas incorrectly included in the mask) often led to the loss of significant portions of the original object, reducing overall mask quality.

Furthermore, peak detection in histograms proved to be very complicated due to varying object lighting conditions, shadows, and reflections. These factors can significantly alter the appearance of the object's histogram across frames, making it impossible to reliably identify and separate object and background regions based solely on histogram peaks.

While unsuitable for direct mask refinement, histogram analysis could be applied at the point extraction stage. By filtering out points from regions with significant histogram deviations, only points on the object could be retained, though at the expense of losing some valid points.

8.2 Detecting Contour Detection Failures

To identify instances of contour detection failure, a method utilizing histogram correlation between successive frames was put into practice. This involved calculating the correlation between the histogram of the masked region in the preceding frame and the current frame, yielding a measure of invariance. High correlation indicates that the mask remains consistent with the object, while a sharp drop suggests that the mask has drifted or failed to capture the object correctly.

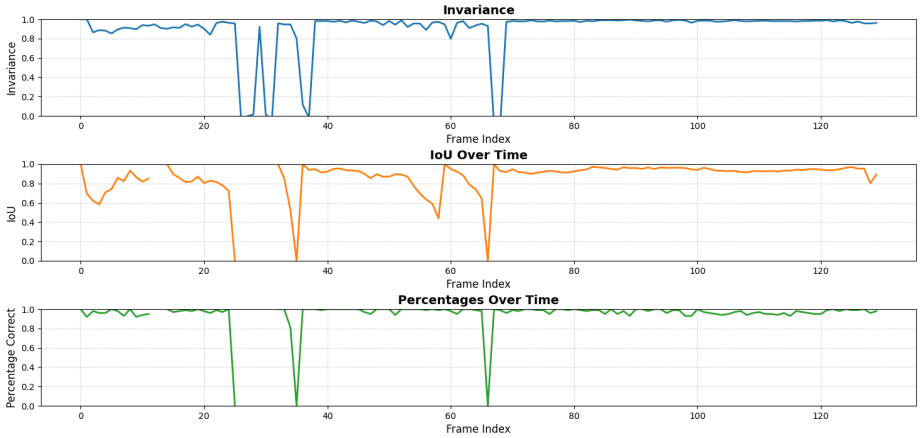


Figure 13: Histogram correlation between consecutive frames. A sharp drop indicates mask failure.

This approach proved highly effective. The correlation metric remained stable when the mask accurately represented the object but exhibited a noticeable decrease when the object was lost (as seen in Figure 13), providing a reliable indicator of contour detection failure in real-time settings.

9 A New Approach

Due to the limitations of traditional contour detection methods, such as sensitivity to lighting variations and low-contrast environments, a more robust solution for object segmentation in our gaze-based object tracking pipeline was sought. This led us to adopt the Segment Anything Model 2 (SAM2), a state-of-the-art foundation model developed by Meta AI for promptable visual segmentation in images and videos. SAM2's ability to generalize across diverse objects without requiring object-specific training, combined with its advanced video tracking capabilities, makes it an ideal solution for enhancing the reliability of our neuroprosthetic control system.

9.1 Segment Anything Model 2 (SAM2)

SAM2 [8] is designed to segment any object in images or videos based on user-provided prompts, such as points, bounding boxes, or masks, addressing the Promptable Visual Segmentation (PVS) task. Building on the original Segment Anything Model (SAM) [6], SAM2 introduces significant improvements in accuracy, speed, and real-time video processing, making it suitable for dynamic, real-world scenarios like those in the Grasping In The Wild (GITW) dataset.

9.1.1 Architecture and Key Components

SAM2 employs a transformer-based architecture optimized for sequential data processing, enabling both single-frame image segmentation and multi-frame video segmentation. Its key innovation is a streaming memory mechanism that ensures temporal consistency in video processing. The architecture, illustrated in Figure 14, includes:

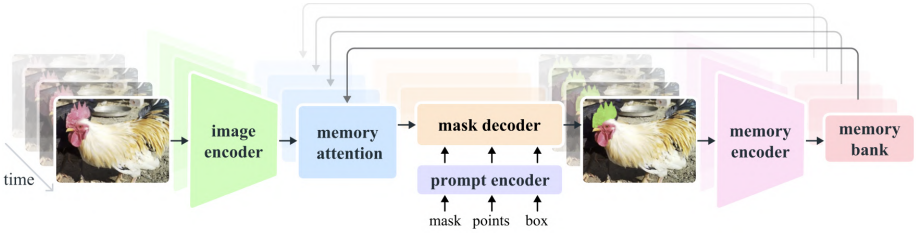


Figure 14: SAM2 architecture overview, from [8].

- **Image Encoder**: Utilizes a pre-trained Hiera transformer to generate unconditioned feature embeddings for each frame, processed once for real-time efficiency.
- **Memory Encoder and Memory Bank**: Extracts and stores features from previous frames' predictions and prompts, maintaining a temporal repository of object information.

- **Memory Attention Module**: Conditions current frame features on past memories via cross-attention, ensuring consistent segmentation of the target object across frames.
- **Occlusion Head** (not visible on figure): Predicts the likelihood of an object being occluded or absent, enabling robust tracking during temporary occlusions or frame exits.
- **Prompt Encoder and Mask Decoder**: Encodes prompts (points, boxes, or masks) and decodes them with frame embeddings to produce segmentation masks, supporting iterative refinement.

These components enable SAM2 to handle complex scenarios, such as occlusions, rapid object motion, and cluttered backgrounds, which are prevalent in the GITW dataset and critical for real world applications.

9.1.2 Modes of Operation

SAM2 operates in two modes tailored to different input scenarios:

1. **Image Segmentation Mode**: Segments objects in a single frame using prompts, very similar to SAM, however offering 6× faster processing and slightly higher accuracy according to [8].
2. **Video Segmentation Mode**: Tracks objects across video frames using the streaming memory mechanism, maintaining temporal consistency and handling occlusions with help from the occlusion head. This mode could be excellent for our pipeline's real-time tracking requirements.

9.1.3 Video Tracking with Memory

In video segmentation mode, SAM2's memory mechanism ensures robust tracking:

- **Feature Extraction**: The memory encoder extracts high-level features from each frame's mask prediction and image embedding.
- **Feature Storage**: Features are stored in a memory bank as a First-In-First-Out (FIFO) queue, retaining up to N recent frames and M prompted frames, along with object pointers for semantic information.
- **Attention-Based Tracking**: The memory attention module correlates current frame features with stored memories, ensuring consistent object tracking.

This mechanism outperforms traditional contour detection by maintaining tracking continuity in challenging scenarios, such as partial occlusions or rapid movements, which are common in egocentric videos like those in the GITW dataset.

10 Integrating SAM2 into the Tracking Pipeline

To incorporate the Segment Anything Model 2 (SAM2) into our gaze-based object tracking pipeline, two integration strategies were explored: using SAM2 as a replacement for contour detection in the mask correction step and employing SAM2's built-in video tracking mode as a complete replacement for the homography-based tracking process. Both approaches leverage SAM2's robust segmentation capabilities but differ in computational complexity and suitability for dynamic scenarios. Below, both method, its implementation, advantages, disadvantages, and evaluation results are detailed.

10.1 SAM2 as a Replacement for Contour Detection

10.1.1 Method

This approach integrates SAM2 into the existing pipeline by replacing traditional contour detection algorithms (e.g., GrabCut) with SAM2 for refining the approximate mask. The pipeline, illustrated in Figure 15, consists of the following steps:

1. **Initial Mask**: The pipeline starts with an initial binary mask M_0, provided by the gaze-guided object detection phase, as described in Section 2.
2. **Homography Approximation**: For each subsequent frame, a homography transformation, computed using ORB keypoints, warps the previous mask M_{n-1} to produce an approximate mask MA_n.
3. **SAM2 Correction**: SAM2 refines MA_n by using the approximate mask as a prompt in its image segmentation mode. The refined mask M_n is then used for tracking in the next frame and for point extraction.

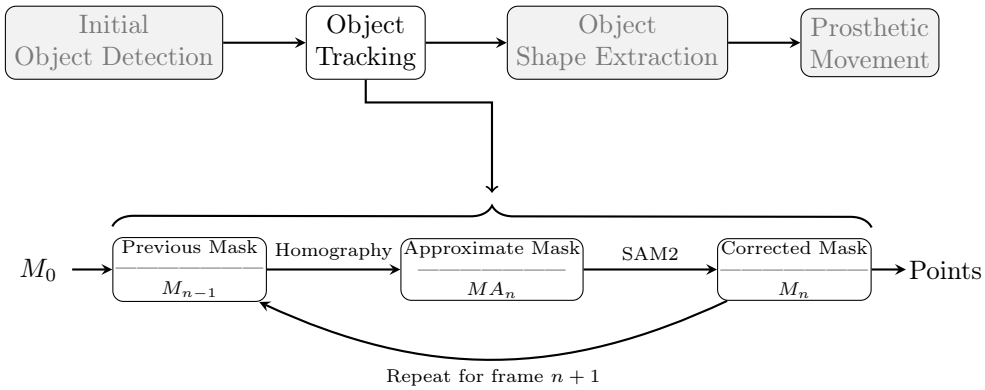


Figure 15: Pipeline using SAM2 for mask correction, replacing contour detection.

SAM2's image segmentation mode offers robustness to lighting variations and occlusions, leveraging its transformer architecture and pre-training on the SA-V dataset (50.9K videos, 642.6K masklets). With GPU acceleration SAM2 can achieves real-time performance, making it suitable for live applications requiring precise mask refinement. Since the homography is already computed for other pipeline components (not covered in this report), this approach incurs minimal additional computational cost for the transformation step.

10.1.2 Evaluation

To evaluate this approach, SAM2 correction was tested every 10 frames on the GITW dataset's *PlatePlace1Subject2* sequence, measuring Intersection over Union (IoU) against ground truth masks. Figures 16 and 17 shows the results.

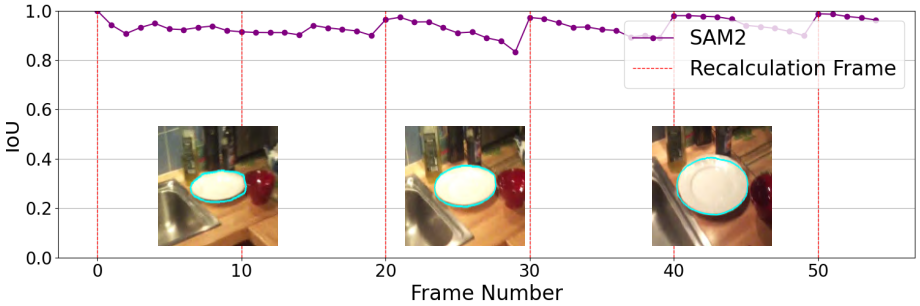


Figure 16: IoU with SAM2 correction every 10 frames on *PlatePlace1Subject2*.

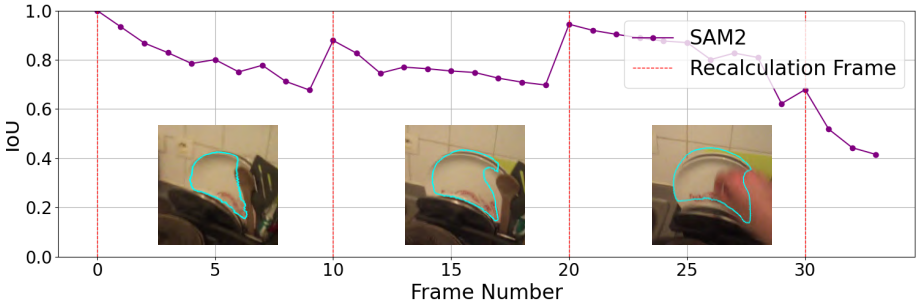


Figure 17: IoU with SAM2 correction every 10 frames on *PlatePlace7Subject2*.

SAM2 achieved mean IoU scores of 0.93 and 0.78 in simple and complex scenarios, respectively. In simple scenarios, SAM2 and GrabCut performed similarly (0.93 vs. 0.92). However, in challenging scenes with cluttered backgrounds, occlusions, or poor lighting, SAM2 significantly outperformed GrabCut (0.78 vs. 0.58). The method reached over 25 FPS on an NVIDIA RTX 4070, meeting real-time requirements. Adjusting the correction frequency (e.g., every

N = 5 or N = 15 frames) allows a trade-off between accuracy and speed: more frequent corrections increase IoU but reduce FPS.

However, this approach has limitations, notably it's reliance on homography restricts its effectiveness for moving objects. Additionally, using SAM2 in image segmentation mode treats each frame independently, neglecting its video tracking capabilities, such as the memory attention module, which could enhance temporal consistency.

10.2 SAM2 as a Full Replacement

10.2.1 Method

The second approach replaces the homography-based tracking entirely with SAM2's video segmentation mode, leveraging its built-in tracking capabilities. The simplified pipeline, shown in Figure 18, is:

1. **Initial Mask**: An initial mask M_0 is provided by the object detection phase.
2. **SAM2 Video Tracking**: SAM2 tracks the object across frames using its streaming memory mechanism, producing masks M_n for each frame.
3. **Point Extraction**: Points are extracted from the masks for 3D shape reconstruction.

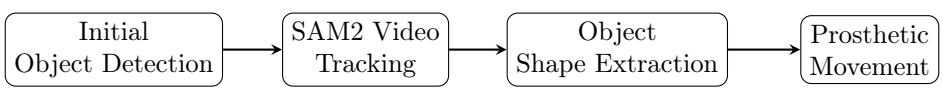


Figure 18: Pipeline using SAM2's video tracking mode as a full replacement.

SAM2's video segmentation mode utilizes its memory encoder and bank to maintain temporal consistency across frames, even for moving objects or during occlusions. However, SAM2 does not natively support live video feeds, requiring a complete video upfront for processing. To enable real-time tracking, a modified version of SAM2 from [11] was adopted, which processes incoming frames incrementally, simulating a live feed. This modification allows SAM2 to operate in our pipeline without pre-loading the entire video, making it suitable for our applications. This approach also eliminates the need for homography in the tracking step, removing any problems linked to having a planar transformation.

10.2.2 Evaluation

SAM2's video tracking mode was evaluated on the *PlatePlace1Subject2* sequence, measuring IoU. Figures 19 and 20 presents the results.

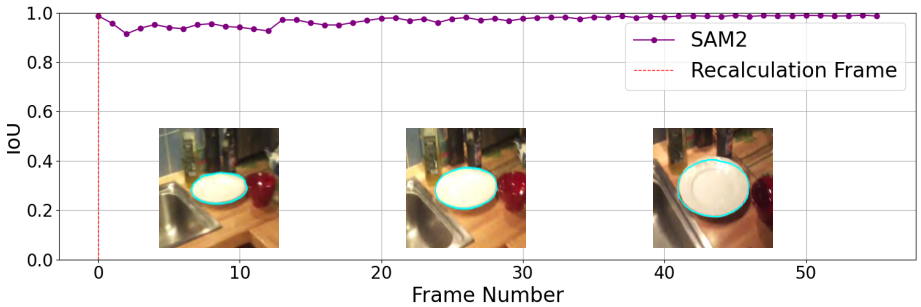


Figure 19: IoU using SAM2 video tracking on *PlatePlace1Subject2*.

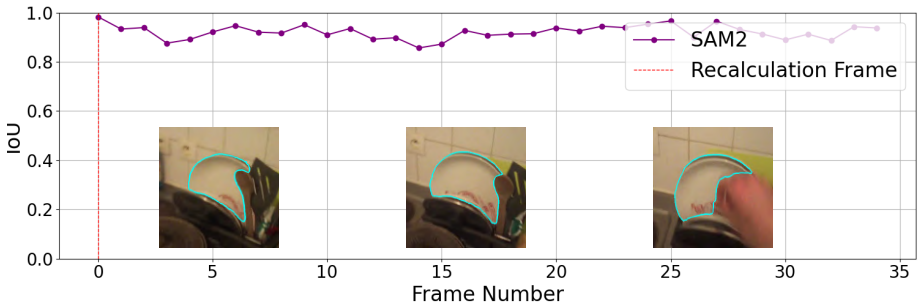


Figure 20: IoU using SAM2 video tracking on *PlatePlace7Subject2*.

This method achieved a mean IoU of 0.97 and 0.92 respectively, consistently higher than the correction-based approach because every frame is being analyzed and due to SAM2's memory attention module. It performed robustly with moving objects and occlusions, unlike the homography-dependent method. However the modified SAM2 implementation is more computationally demanding and runs at 16 FPS on an NVIDIA RTX 4070, which could be low for real-time applications and can't easily be speed up without better hardware unlike the correction based approach.

The primary drawback is the increased computational overhead, which may strain resource-constrained devices. However, this could be mitigated by only processing certain frames, but this remains to be tested. Still SAM2's video mode is near perfect for real time tracking and segmentation if you have the right hardware.

11 Conclusion

This internship at the MIA Laboratory, in collaboration with LABRI, INCIA, Pollen Robotics, and UGECAM, focused on developing a gaze-based object tracking system for upper limb prostheses, emphasizing the object tracking phase. The project successfully advanced the I-WRIST project's pipeline by implementing and evaluating homography-based tracking with traditional contour detection (e.g., GrabCut) and integrating the state-of-the-art Segment Anything Model 2 (SAM2) for enhanced segmentation and tracking. Two datasets, Grasping In The Wild (GITW) and Cylinder-Clothpin, provided diverse scenarios for algorithm development and evaluation, with performance assessed using Intersection over Union (IoU) and precision metrics. Point extraction methods (grid-based and random sampling) were explored to support 3D shape reconstruction, though further evaluation is needed.

Three tracking approaches were compared:

- **Contour Detection**: This method is a viable option when CUDA is unavailable, offering simplicity and reasonable performance for static or slow-moving objects in controlled environments. However, it struggles with complex scenes due to sensitivity to lighting and low-contrast conditions.

- **SAM2 Correction**: Replacing contour detection with SAM2 in the homography-based pipeline achieves higher speed when hardware acceleration is available, and better accuracy. However, reliance on homography limits its effectiveness for moving objects, and it underutilizes SAM2's temporal tracking capabilities.

- **SAM2 Video Tracking**: Using SAM2's video segmentation mode provides superior robustness for moving objects and occlusions, with higher mean IoU (0.97 vs. 0.93 in simple scenarios; 0.92 vs. 0.78 in complex scenarios). Its computational demand is a drawback but could be mitigated with hardware improvements or using a lighter model.

The SAM2 video tracking mode is preferred for real-world applications due to its robustness in dynamic, cluttered environments like those in the GITW dataset.

Moving forward, the aim is to combine SAM2's video tracking with homography by using SAM2's memory mechanism to segment every N frames and applying homography for intermediate frames to always have a estimation of the object position and shape. This hybrid approach could balance robustness and computational efficiency, leveraging the strengths of both methods. Additionally, integrating the initial object detection, developed by another member of the team in parallel, would streamline the pipeline, and allow for real world testing and benchmarking.

References

[1] Bolutife Atoki, Jenny Benois-Pineau, Renaud Péteri, Fabien Baldacci, and Aymar de Rugy. Object segmentation in the wild with foundation models: application to vision assisted neuro-prostheses for upper limbs. 2025.

[2] Tony Chan and Luminita Vese. An active contour model without edges. In Mads Nielsen, Peter Johansen, Ole Fogh Olsen, and Joachim Weickert, editors, *Scale-Space Theories in Computer Vision*, pages 141–151. Springer. doi:10.1007/3-540-48236-9_13.

[3] Rami Cohen. The chan-vese algorithm. URL: https://arxiv.org/abs/1107.2782v2.

[4] Pascal Getreuer. Chan-vese segmentation. 2:214–224. URL: https://www.ipol.im/pub/art/2012/g-cv/, doi:10.5201/ipol.2012.g-cv.

[5] Michael Kass, Andrew Witkin, and Demetri Terzopoulos. Snakes: Active contour models. 1(4):321–331. doi:10.1007/BF00133570.

[6] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. URL: https://arxiv.org/abs/2304.02643v1.

[7] Pablo Márquez-Neila, Luis Baumela, and Luis Alvarez. A morphological approach to curvature-based evolution of curves and surfaces. 36(1):2–17. URL: https://ieeexplore.ieee.org/document/6529072, doi:10.1109/TPAMI.2013.106.

[8] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. URL: https://arxiv.org/abs/2408.00714.

[9] Carsten Rother, Vladimir Kolmogorov, and Andrew Blake. "GrabCut": interactive foreground extraction using iterated graph cuts. In *ACM SIGGRAPH 2004 Papers*, pages 309–314. ACM. URL: https://dl.acm.org/doi/10.1145/1186562.1015720, doi:10.1145/1186562.1015720.

[10] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. ORB: An efficient alternative to SIFT or SURF. In *2011 International Conference on Computer Vision*, pages 2564–2571. ISSN: 2380-7504. URL: https://ieeexplore.ieee.org/document/6126544, doi:10.1109/ICCV.2011.6126544.

[11] Guiyong Zheng. Gy920/segment-anything-2-real-time. original-date: 2024-08-01T15:19:31Z. URL: https://github.com/Gy920/segment-anything-2-real-time.